

S403011 Machine Learning Project: Weather Prediction

Team: Power Rangers

Authors: Katrin Kulberg, Eric Lee, Rafael Pereira Tinoco

Introduction

Weather, being a central element of the natural environment, has a major impact on everyday life, starting from simple daily decisions, like choosing what to wear, to extreme events that affect agriculture and the overall well-being of society, such as heatwaves, floods and storms. According to the [National Centre for Atmospheric Science \(NCAS\) \(2025\)](#), weather can be defined as the state of the atmosphere at a given time, described by characteristics such as temperature, precipitation, and wind.

Due to its fundamental role in human life, weather forecasting has been a topic of interest for centuries and continues to be fascinating today, where it is more than ever relying on big data and computational methods. The aim of this project is to develop different machine learning models for predicting the temperature in Bern, the capital of Switzerland, at 12-, 24-, and 48-hour horizons, and compare their performance in order to identify the model providing the most accurate temperature forecasts.

1 Background

1.1 Weather Prediction History and Current Trends

The following subsection gives a brief overview of the history of weather prediction, which is written based on [Lynch \(2008\)](#), and more recent developments involving machine learning methods, which are described based on [Zhang et al. \(2025\)](#).

The field of weather forecasting has seen many changes over the last 150 years. Starting from subjective, purely empirical approaches by observing meteorological parameters such as pressure, the first major breakthrough was made in the 1920s by the English mathematician Lewis Fry Richardson, who was the first to attempt modelling the future state of the atmosphere by numerically solving the underlying differential equations. While his idea was not practically feasible at the time due to limitations in data, computational power and mathematical techniques, it provided a cornerstone for the advancements in the numerical weather prediction that would follow in the next decades. For example, the first successful numerical weather forecast, produced in 1950 by the American meteorologist Jule Gregory Charney and his team, was, with some key mathematical refinements and the help of a computer, largely based on Richardson's idea.

This first success in 1950 left a strong impression on the scientific community and raised widespread interest in developing numerical methods for weather prediction. Since then, many new models have been

proposed, which not only incorporate more sophisticated physics and mathematical rigour compared to the earlier methods, but are also ever more dependent on computational power.

While modern weather forecasting is still predominantly based on numerical weather prediction models, which rely directly on the underlying physical principles, machine learning methods are becoming increasingly popular in the field as a complement to traditional models. Most notably, thanks to their ability to recognise patterns and relationships in large quantities of data, which may remain undetected by traditional techniques. Numerous studies have shown that, for simpler tasks, traditional machine learning models such as random forest or support vector machines are often sufficient, while, for more difficult situations involving high-dimensional data, neural networks tend to be more effective.

Although machine learning-based weather prediction has demonstrated promising results, similarly to traditional methods, it also struggles in accurately predicting extreme events. This limitation is partly data-driven, as extremes appear in the training data rarely, but also model-driven, since making predictions beyond the observed range is inherently challenging. Furthermore, as models are generally optimised for average performance, the tails of the distribution tend to be predicted less accurately.

In addition to extreme events, machine learning methods also often suffer from a lack of explainability. However, there is ongoing research aiming to mitigate this issue, with two of the most promising directions being physics-informed neural networks, which utilise the underlying physical principles, and explainable artificial intelligence, which provides insights into the model's decision-making process. Despite these limitations, the role of machine learning methods should not be underestimated, and it is anticipated that combining them with existing weather forecasting systems could complement the latter and lead to a more effective weather prediction methodology overall.

1.2 Climate of Switzerland and Weather Conditions in Bern

In the following, the overview of Switzerland's general meteorological patterns are described based on [MeteoSwiss \(2025b\)](#), while the weather conditions in Bern are described based on climate normals from 1991 to 2020 according to [MeteoSwiss \(2025a\)](#).

Despite its relatively small size, Switzerland has a very diverse climate, largely due to its geography. The most influential features are the Alps, along with the Jura mountains in the northwest and the Swiss Plateau in the central part of the country. Directly driven by its geography, higher temperatures usually occur in lower-lying areas in Western and Northern Switzerland and particularly in the southern tip of the country near the lakes of Ticino.

Precipitation is generally higher in the mountainous areas, such as the Alps and Jura, with the exception of inner-Alpine valleys, which are very dry. Moreover, on the southern side of the Alps, precipitation also tends to be higher, with the heaviest events recorded in Ticino. It has also been observed that precipitation tends to be higher in summer compared to winter months. When it comes to snow, above altitudes of 1200-1500 metres above sea level, precipitation during winter usually falls as snow. In

contrast, in low-lying areas, such as Western Switzerland and the Ticino region, snowfall during winter is much less common, and snow-free winters can occur fairly often.

Another characteristic of Swiss weather is fog, which is more common during colder months and can lead to the formation of low-lying stratus clouds. Such clouds form when fog is lifted by cool and dry, usually easterly or northeasterly winds, as a result forming a cloud cover, which can, depending on the wind strength, remain in place for a prolonged period. Low-lying stratus clouds occur frequently in the northern part of central Switzerland and in the southern foot of the Jura, explaining, for instance, why the relative sunshine duration in those regions tends to be lower.

Wind conditions are also highly location-specific. Near the surface, the winds are usually blowing from westerly to southwesterly or northwesterly to northeasterly directions. In general, lower-lying areas are not particularly windy, with annual average wind speeds staying below 10 km/h, while higher elevations, which are more exposed, frequently experience stronger gusts. Another wind-related characteristic of Swiss weather is the existence of Foehn winds, which occur on both sides of the Alps, being thus called northern and southern Foehns, respectively. This phenomenon occurs when an air mass crosses over the mountain range and descends on the opposite side of the slope as a warm, dry wind. Foehns are more likely to occur during the colder months, with the northern Foehn bringing relatively mild and clear weather conditions to the south side of the Alps, and the southern Foehn bringing similar conditions to the northern part of the country.

Bern, located in the central part of the Swiss Plateau at an altitude of around 550 metres above sea level, experiences a moderate climate with distinct seasons. Summers are relatively warm, with monthly average maximum temperatures reaching up to 25 degrees, which give rise to occasional heatwaves. In contrast, winter temperatures often fall below freezing, and snowfall is not particularly uncommon, though the snow cover usually remains slim.

Due to its location on the Swiss Plateau, which is prone to the formation of low-lying stratus clouds, Bern experiences many cloudy days per year, especially during the colder months. Regarding precipitation, average monthly amounts generally remain below 120 millimetres, and heavier rainfall usually occurs during the summer season. An opposite annual pattern is observed in relative humidity, with lower values in spring and summer, annually ranging between 70% and 90% on average. Finally, concerning winds, they are generally light, with monthly averages between 5 and 8 km/h.

2 Overview of Data

Data used in this project originates from 10 weather stations across Switzerland. In particular, the measurements were collected from the following locations (see Figure [2.1](#)): Andermatt (ANT), Basel (BAS), Davos (DAV), La Dôle (DOL), Genève (GVE), Interlaken (INT), Lugano (LUG), Sion (SIO), St. Gallen (STG), and Zermatt (ZER).

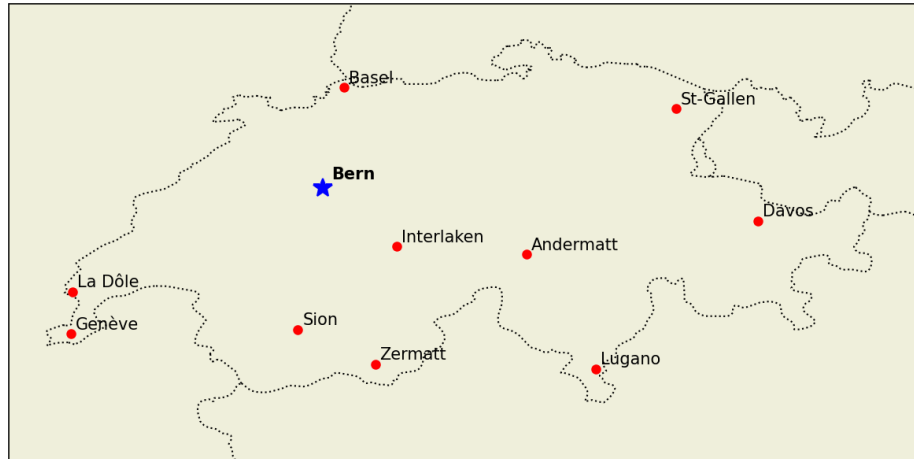


Figure 2.1: Location of weather stations, with Bern being marked as reference. Map is based on station coordinates obtained from [MeteoSwiss \(2025c\)](#).

At each station, the following parameters were measured:

- **fkI010h0** Wind speed (hourly mean, m/s);
- **fkI010h3** Gust peak (3-second hourly maximum, m/s);
- **gre000h0** Global radiation (hourly mean, W/m^2 (watts per square meter));
- **prestah0** Atmospheric pressure at barometric altitude (QFE) (hourly mean, hPa);
- **pp0qffh0**¹ Atmospheric pressure reduced to sea level (QFF) (hourly mean, hPa);
- **rre150h0** Precipitation (hourly total, mm);
- **sre000h0** Sunshine duration (hourly total, minutes)
- **tre200h0** Air temperature 2 metres above ground (hourly mean, °C);
- **ure200h0** Relative air humidity 2 metres above ground (hourly mean, %).

In addition, the season (Spring, Summer, Autumn, Winter) and the time of the day (in hours, ranging from 0 to 23) of each measurement were recorded. The dataset also includes information about the temperature at the time of measurement in Bern, as well as its 24-hour lagged counterpart, i.e. the temperature recorded 24 hours earlier.

In the following sections, it is important to differentiate between the dataset training, which includes all described variables as well as the temperatures in Bern at 12-, 24-, and 48-hour horizons (i.e. the

¹Measured only in the following five stations: BAS, GVE, INT, LUG, and SIO.

response variables), and the dataset `test`, which contains all predictors but no response variables. All models are trained and validated using the dataset `training`, while the dataset `test` is reserved solely for making predictions for the Kaggle competition. Accordingly, all discussions regarding data in the following sections refer to the dataset `training`, unless stated otherwise.

2.1 Further Considerations

As the first step, outliers and missing values were considered. Regarding the outliers, no suspicious records were detected, as all measurements fell within the plausible ranges. It is worth mentioning that, during this step, both the absolute measurement values and the locations of the weather stations were considered. The latter is particularly relevant to be able to account for potential extreme values that may stem from geographic characteristics of specific stations, such as aggressive wind gusts or heavy rainfall, which could be otherwise seen as outlier at first glance. When it comes to missing values, their total number relative to the size of the full dataset was low, being only 150 of 7579 records (around 2%). Therefore, no further consideration was given to them, and rows containing missing values were simply discarded from further analysis.

To account for the periodic nature of temperatures over the year and throughout the day, the season and hour of the day were transformed into numeric form by using sine and cosine functions. The new variables were defined as follows:

$$s_{\sin} = \sin\left(\frac{2\pi \cdot s}{4}\right), \quad s_{\cos} = \cos\left(\frac{2\pi \cdot s}{4}\right), \quad h_{\sin} = \sin\left(\frac{2\pi \cdot h}{24}\right), \quad h_{\cos} = \cos\left(\frac{2\pi \cdot h}{24}\right),$$

where s denotes the season (0 for Spring, 1 for Summer, 2 for Autumn, and 3 for Winter), and h represents the hour of the day (0 to 23). Here, it should be noted that even though season is only recorded as four distinct values and therefore sine and cosine representations are sparse, the inherent cyclical pattern preserved in the variables is still likely to be helpful for modelling.

An additional transformation that can reduce the correlation between some of the input variables is made possible by the underlying physics. The relative humidity, U , is defined as the ratio between the partial water vapour pressure in the atmosphere, P_w , and the saturation water vapour pressure, $e^{(\text{sat})}$, which is known to be a function of both temperature and pressure:

$$U = \frac{P_w}{e^{(\text{sat})}(T, P)}.$$

The free variable would be the partial pressure, and thus this would be the desirable input for the models to minimise confusion during the regression, rather than the relative humidity itself. A common parametrisation for the saturation pressure in air is given by [Buck \(1981\)](#):

$$e^{(\text{sat})}(T, P) = e(T) \cdot f(T, P),$$

where $e^{(\text{sat})}$ is the saturation pressure, T is the temperature, P is the pressure, e is the saturation pressure of pure water vapour and f is the so-called enhancement factor, to act as a correction for the fact that the desired quantity is the saturation in air. For both, empirical parametrisations are provided:

$$e(T) = a \cdot \exp\left(\frac{(b - \frac{T}{d})T}{T + c}\right) \quad f(T, P) = 1 + A + P \cdot (B + C \cdot (T + D + E \cdot P)^2)$$

For the relevant temperature ranges for atmospheric phenomena, [Buck \(1981\)](#) offers the following coefficients, as presented in Table 2.1.

Table 2.1: Arden Buck equation coefficients.

A	B	C	D	E	a	b	c	d
4.1×10^{-4}	3.48×10^{-6}	7.4×10^{-10}	30.6	-3.8×10^{-2}	6.1121	18.729	257.87	277.3

Given the set of measurements available at each station, the partial water vapour pressure can be obtained directly from (previous equation for U):

$$P_w = U \cdot e^{(\text{sat})}(T, P) = U \cdot e(T) \cdot f(T, P) = U \cdot a \cdot \exp\left(\frac{(b - \frac{T}{d})T}{T + c}\right) \cdot [1 + A + P \cdot (B + C \cdot (T + D + E \cdot P)^2)]$$

The model will then receive this calculated value, for every station, instead of the relative humidity.

3 Descriptive Overview

The distribution of the target variables, specifically the temperatures in Bern at the 12-, 24-, and 48-hour horizons, is illustrated in Figure 3.1. As can be seen in the plot, all three distributions are relatively similar and resemble the shape of a normal distribution, with mild positive skewness caused by slightly heavier right tails. When it comes to mean and standard deviation, they are, for all three time horizons, around 10 °C and 8 °C, respectively.

Since season is inherently an important factor for temperature prediction, and to better illustrate seasonal variation, the distribution of temperatures in Bern throughout the year is depicted in Figure 3.2. As expected, the highest temperatures are recorded during summer, with a maximum of around 34 °C, and the lowest during winter, with a minimum of around −14 °C. Spring and autumn are relatively similar, with mean temperatures of around 10 °C for both seasons.

Another key factor in temperature forecasting is the hour of the day, for which distributions by season are also shown in Figure 3.2. It is evident that the effect of the hour is cyclical, making modelling it by sine and cosine oscillations a natural choice. It is also worth noting that the hourly effect is stronger during warmer periods, whereas in winter, when temperatures are lower and more stable throughout the day, the daily temperature curve is less pronounced.

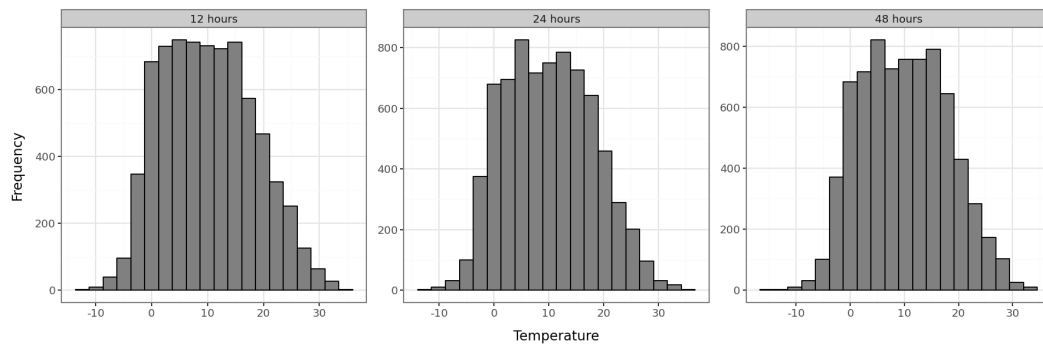


Figure 3.1: Distribution of temperatures at the 12-, 24-, and 48-hour horizons.

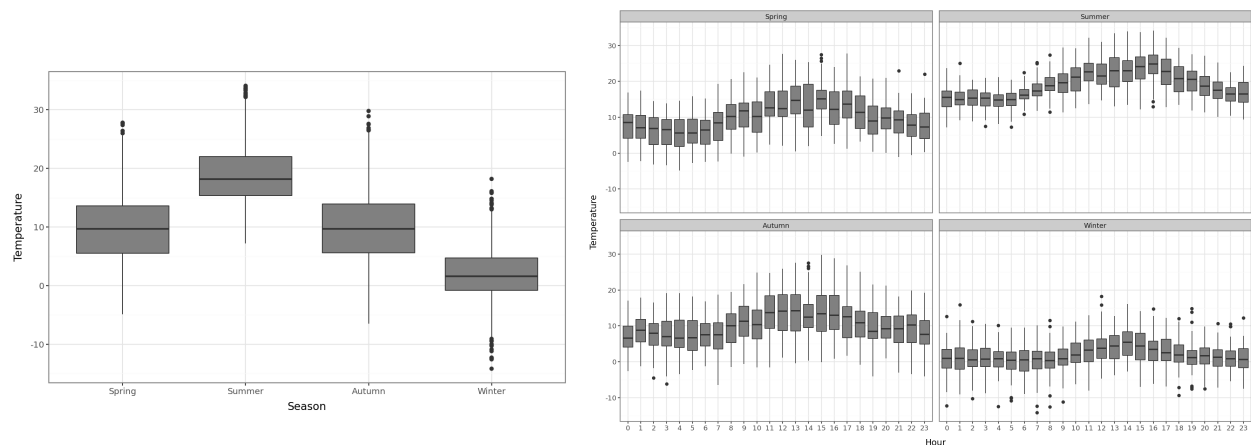


Figure 3.2: Distribution of temperatures by season (left) and hourly temperatures by season (right) in Bern.

To provide a broader overview of the dataset, the linear pairwise correlations between all variables are illustrated in Figure 3.3. The heatmap shows several highly positively correlated clusters between stations, particularly for global radiation levels, temperature, atmospheric pressure, both at barometric altitude and at sea level, and for the partial water vapour pressure. This matches physical intuition, as the measured quantities reflect regional phenomena that affect multiple stations. Moreover, hourly average wind speed and the three-second gust peaks at the same station are also strongly correlated, as would be expected given that they measure the same underlying phenomenon. In contrast, consistent negative correlations, though not as strong as some of the strongest positive ones, are observed between pressure and the wind speed. This observation is also consistent with intuitive expectations, as higher wind speeds tend to occur in lower-pressure conditions.

The features that are most strongly linearly correlated with response variables are also worth mentioning. Specifically, they are the temperature in Bern at the time of measurement and its lagged counterpart, temperatures from other stations across the country, and lastly partial water vapour pressure. It should be noted that the correlations between features and the temperature 12 hours later are slightly different

from the correlations with the other two response variables. This phenomenon may be explained by the fact that a 12-hour shift in time often corresponds to a transition between day and night, which can blur the relationships between atmospheric characteristics and temperature. Conversely, at the 24- and 48-hour horizons, the temperature is measured at the same time of day as the original observation, and thus the relationships remain more consistent.

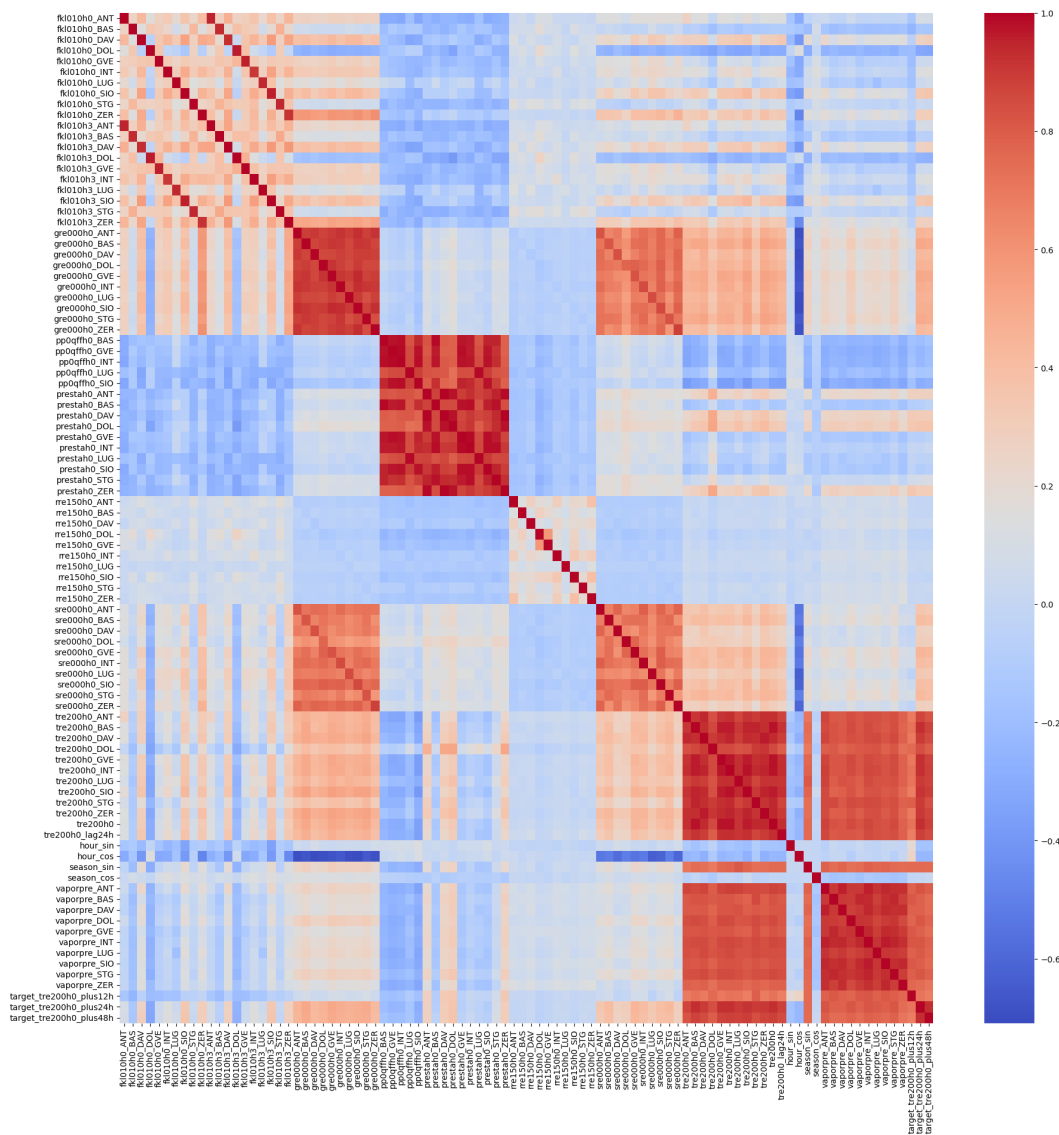


Figure 3.3: Linear pairwise correlations among all variables in the dataset.

The different meteorological parameters were also explored individually in order to understand their distributions and inherent particularities that could be leveraged for modelling purposes. A selection of the most significant variables in this regard is presented in Figure 3.4, with each corresponding to measurements from the Basel station as an example. It can be observed that global radiation and

precipitation have L-shaped distributions, with most of the mass being concentrated near zero. Hourly sunshine duration exhibits a bimodal behaviour, where most measurements correspond to either 0 or 60 minutes. As the two variables describing wind conditions have nearly identical distributions, only one of them (the average hourly wind speed) is shown here, which resembles a Gamma distribution, with a long right tail. Taking everything into account, it is clear that none of these variables are normally distributed. This means that the basic standardisation approach of shifting and scaling all the variables based on their estimated mean and variance so that the inputs to the models are all of comparable magnitude will be problematic for these particular meteorological parameters, as the estimated variance and mean would still result in values that are significantly away from being centered around zero with magnitudes not much greater than 1. Furthermore, the strictly non-negative nature of the underlying quantities may be meaningful for correctly modelling their non-linear effects on the response variable, and thus should be taken into account during scaling.

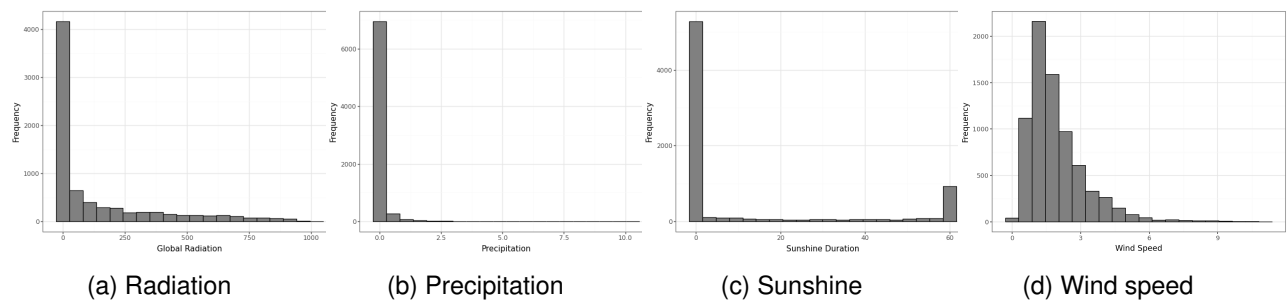


Figure 3.4: Distributions of global radiation, precipitation, sunshine duration, and wind speed at the Basel station.

4 Machine Learning Methods for Weather Prediction

Before proceeding to the discussion of the specific approaches, it is important to note that, prior to modelling, the dataset (i.e. the dataset `training` mentioned in Section 2) was split into two parts, with 80% of the data being used for training and the remaining 20% being left for internal validation. Moreover, as scaling the input data is important for the accuracy of the models that will be considered, this step is also worth mentioning. In order to maintain the separation between training and validation data, only the former was taken into account when determining the constants for scaling the data used in the assessment within this section. However, for the Kaggle competition, and for consistency with the fact that the entire dataset will be used to train the model, scaling is also based on all datapoints.

Following the observations from the previous section, for variables with large deviations from normality and strictly positive values (i.e. wind speed, sunshine duration, precipitation, and global radiation), scaling is performed based on the maximum absolute value, which transforms the variables to a range between 0 and 1, thus preserving their strict positivity. For the remaining variables (atmospheric pressure, temperature, including temperatures in Bern, and partial water vapour pressure), the more common standardisation that subtracts the estimated mean and divides by the estimated variance is applied. It is

worth noting that no scaling is used for sine and cosine encodings describing the cyclical behaviour of the hour of the day and season of the year.

In this project, the metric of performance is the mean absolute error (MAE), which is defined as follows:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

where y_i denotes the true value of the response, $\hat{y}_i$ its predicted value, and n corresponds to the number of data points. This metric will be used consistently for hyperparameter optimisation, as well as for evaluating the performance of the models. While MAE was chosen primarily to remain consistent with the Kaggle competition, it should be noted that it is quite a good choice overall, as it reflects the average behaviour of a model while being relatively robust to outliers.

4.1 Basic Models

While the selection of machine learning models that could be applied to the weather prediction problem at hand is fairly diverse, this project only considers the following four: linear regression, random forest, support vector regression (SVR), and tree-based boosting. It should be pointed out that linear regression is considered primarily as a baseline, since it is unlikely that modelling such a complex phenomenon as weather while relying only on simple linear relationships will be sufficient. On the other hand, the remaining models are much more likely to exhibit greater predictive performance and are therefore of direct interest. It is also worth mentioning that neural networks are not considered in this project, mainly due to their inherent complexity and black-box nature, which would make it unfeasible to justify many of the decisions behind the design of the model. Furthermore, using such models often requires extensive, often computationally heavy experimentation and substantial prior experience, making them less attractive for this particular project.

In the following, a brief summary of each model is provided, giving an overview of its main characteristics. In addition, the most important hyperparameters are also outlined. The overviews of linear regression, random forest and boosting are written based on [Hastie et al. \(2009\)](#), whereas the description of SVR is based on [MathWorks \(2025\)](#).

A linear regression model is a statistical model used in case of a linear relationship between the response variable and the predictors. Stemming from classical statistics, it is one of the simplest and most interpretable models. The model parameters are estimated using ordinary least squares by minimising the sum of squared errors, and the predictions are obtained by considering a linear combination of input features and the estimated coefficients. Contrary to the large majority of machine learning methods, standard linear regression has no tunable hyperparameters.

Random forest is an ensemble-based machine learning model consisting of a collection of decision trees, each of them having a low bias but relatively high variance. To reduce variance, the model makes use of the important principle that the variance of the average of independent random variables is smaller

than the variance of each individual variable. Random forest follows this idea by considering a set of decorrelated decision trees, each of them being relatively complex. This decorrelation is achieved through bootstrap samples of the data and by selecting a random subset of m predictors at each split in the tree, with m being one of the most important hyperparameters for the random forest. In the end, predictions from all trees are averaged. Some other significant hyperparameters include the number of total trees, as well as the parameters describing the complexity of each individual tree.

Another popular ensemble-based method is boosting. Although the idea of boosting can be applied to many machine learning models, one of the most common implementations in practice uses decision trees. Contrary to random forest, which makes use of relatively complex trees to reduce variance, boosting for decision trees is based on combining many weak models, each of them having high bias and low variance. These models are fitted sequentially, with each new model trying to correct the errors of the previous, as a result reducing the overall bias. In the end, predictions of all sub-models are summed. The most critical hyperparameters for tree-based boosting are the number of trees, the learning rate, and the parameters describing the complexity of each individual weak learner.

Support Vector Regression is an extension of the Support Vector Machine (SVM) framework for predicting continuous values. In simple terms, SVR fits a linear hyperplane in a chosen feature space that approximates the data while controlling for a tolerance parameter ε , which defines the so-called ε -tube around the regression function. This regression function can be linear or non-linear in the original input space, depending on the kernel used. The significance of the ε -tube becomes apparent in the optimisation problem, as deviations smaller than ε are ignored, while the larger ones are assigned a penalty. Therefore, ε can be seen as a tuning parameter, with larger values making the regression function less sensitive, and smaller values leading to a tighter fit. Finally, the optimisation problem is defined as a trade-off between model complexity and violations of the tolerance, controlled by another hyperparameter C . Depending on the chosen kernel, additional hyperparameters can be considered, such as γ , which determines the width of the Gaussian (RBF) kernel, or the degree for the polynomial kernel.

In this project, all machine learning models except for the tree-based boosting are implemented using the library `scikit-learn` ([Pedregosa et al., 2011](#)). Boosting is implemented with the library `XGBoost` ([Chen and Guestrin, 2016](#)). The primary motivation to use `XGBoost` stems from its advantageous features, as it is better optimised and thus much faster, as well as offers a wider selection of hyperparameters, which in theory allows for achieving better predictive performance in the resulting model.

As the libraries offer a relatively diverse set of tunable hyperparameters for each model, it is also worth discussing some of the specific choices made regarding them. For instance, for SVR, only the RBF kernel was considered, as it is very flexible and therefore suitable for modelling complex non-linear phenomena such as the weather. Moreover, the kernel coefficient γ was fixed to its default value (scale) to mitigate the computational complexity of the tuning procedure, as it was observed that poorly chosen values of γ resulted in very slow model fitting. When it comes to the tree-based methods, all the most significant hyperparameters discussed previously were included, including the total number of trees,

characteristics describing the complexity of each tree, such as maximum depth, as well as the learning rate for boosting. On top of that, for boosting, additional regularisation hyperparameters corresponding to $L1$ and $L2$ norms were considered, which are useful in preventing overfitting.

4.1.1 Tuning the Models

For all models with tunable hyperparameters (i.e., all models excluding linear regression), the optimal hyperparameter combination was determined using k -fold cross-validation. Here, k was set to 7, which was deemed as a good balance between computational efficiency of the tuning procedure and the predictive performance of the resulting model. The optimal set of hyperparameters was selected as the one minimising the average MAE across cross-validation folds.

The hyperparameter optimisation in practice was carried out by employing the `Optuna` library ([Akiba et al., 2019](#)). The more automated approach was primarily selected for its ability to efficiently explore continuous search spaces of hyperparameters by concentrating only on the most promising regions. This makes it feasible to consider a wider selection of hyperparameters and denser ranges, which should result in models with better predictive performance, while keeping the computational cost under control compared to a more direct grid search. An additional advantage is the ability to employ early pruning in the k -fold cross-validation, where, in case the set of hyperparameters does not look promising based on the estimated MAE after $p = 2$ folds, the rest of the folds may be altogether skipped, as a result further decreasing the computational cost of the tuning procedure.

Since the algorithm does not consider each combination explicitly, another important parameter is the number of trials, which essentially determines how thoroughly the hyperparameter space is explored. In this project, the number of trials was set to 100, which was seen as a sufficient trade-off between thorough exploration and computational cost. While it is possible that a greater number of trials might yield a slightly different combination of hyperparameters, it is very likely that the improvements would be marginal.

Two other important decisions when it comes to the usage of the `Optuna` library concern the choice of the sampler, which determines which values of the hyperparameters are evaluated in each trial, and the pruning algorithm. In this project, the default `TPESampler` was employed, which is based on the Tree-structured Parzen Estimator algorithm. This choice is motivated by its relatively good performance, especially given the chosen number of trials, and good support for early pruning. Regarding the pruner, the `WilcoxonPruner` was used. This pruning strategy is based on the non-parametric Wilcoxon signed-rank test, being applied to the comparison of the intermediate results of the current trial with the results of the best trial. Pruning is conducted when there is sufficient statistical evidence at a confidence level $1 - \alpha$ (with $\alpha = 0.1$) that the current trial is likely to perform worse compared to the best completed trial. By relying on statistical evidence across multiple folds, this pruning strategy is likely to be more conservative and mitigates the risk of early pruning.

4.1.2 Results

After finding the optimal combination of hyperparameters for the models that required tuning, all models were trained on the training set and then evaluated on the test set by means of MAE. In addition, MAEs were also computed on the training set to assess the extent of overfitting, which can be quantified as the difference between the training and test errors. The results for all four models are presented in Table 4.1.

Overall, the best performance is achieved by SVR, as the model obtains the lowest MAE scores across all time horizons on the test set. The second best in terms of performance is XGBoost, followed by random forest, and finally linear regression, which, as expected, achieves the lowest predictive accuracy among the more flexible models. However, it is interesting that linear regression attains the best predictive performance at the 24-hour horizon, while for the remaining models, the highest accuracy is observed for 12-hour predictions. This may be because the 12-hour shift in time often corresponds to a transition between day and night, likely introducing additional non-linear relationships. These patterns are more difficult for a simple linear model to capture, whereas the more flexible non-linear models are able to handle them more efficiently.

While the general ranking of the models remains consistent across the three time horizons, the relative differences vary. For example, for 12-hour predictions, the gap between random forest and XGBoost is quite large, around 0.15, whereas for 24-hour and 48-hour predictions, the differences are much smaller, around 0.08 and 0.06, respectively. Similarly, XGBoost is somewhat closer to SVR for 24-hour and 48-hour predictions, with differences being 0.02 and 0.01, respectively, while for the 12-hour prediction, the gap is slightly larger, around 0.08. Since SVR and XGBoost are both relatively flexible models, it is interesting that the gap between the two becomes the most pronounced for the 12-hour prediction. However, this may be due to the applied scaling, which likely benefited the SVR the most in the more challenging 12-hour prediction setting, increasing its advantage over the XGBoost, which as a tree-based model is largely invariant to feature scaling.

Table 4.1: Mean absolute error (MAE) on training and test sets across all time horizons for each model.

Model	12 hours		24 hours		48 hours	
	Train	Test	Train	Test	Train	Test
Linear Regression	2.007	2.075	1.948	1.898	2.547	2.519
Random Forest	0.621	1.724	0.689	1.807	1.028	2.390
SVR	1.102	1.495	1.367	1.708	1.744	2.321
XGBoost	0.099	1.572	0.081	1.730	0.076	2.327

Regarding the relationship between training and test MAE, the largest gaps are observed for the two tree-based models, XGBoost and random forest, with XGBoost showing a particularly large difference between training and test MAE. Despite this, the model achieves a very good accuracy on the unseen test data, suggesting that the extent of overfitting does not pose a problem for its ability to generalise. Among the considered models, the most similar performance across the training and test data is observed for linear regression, which, given its simple nature, is unlikely to overfit in this complex setting. However, it is interesting that for linear regression in particular, the training MAE is actually lower than the test MAE

for 24- and 48-hour predictions, further supporting the conjecture that it is not able to fully capture the dynamics of the underlying phenomena. For all other models, the expected pattern is observed, with the lower MAE being achieved on the training data.

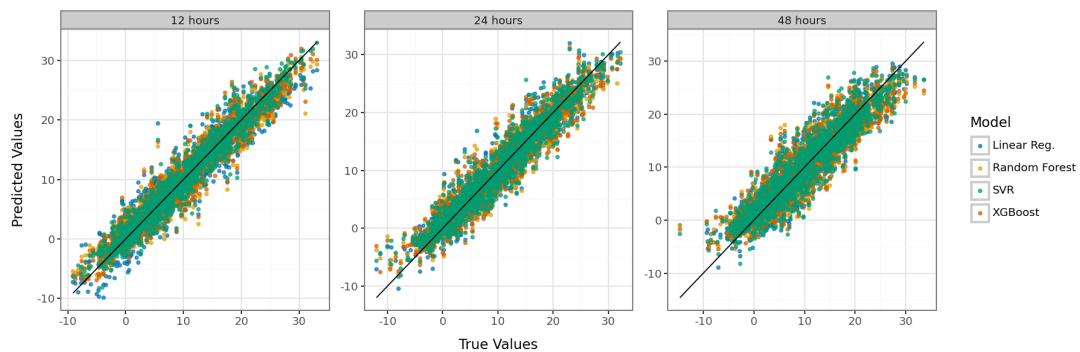


Figure 4.1: Relationship between true and predicted values for each model across 12-, 24-, and 48-hour horizons. The black line represents perfect predictions and serves as a reference.

To further investigate the behaviour of the models, the relationships between the true and predicted temperatures for each model across the three time horizons are shown in Figure 4.1. The plots illustrate one common phenomenon. In particular, the models tend to underpredict at the upper tail, as most points at high temperatures lie under the reference line, which represents perfect predictions, and overpredict at the lower tail, as there the points tend to lie above the reference line. The figure also suggests that SVR indeed tends to perform best, as its predictions generally lie closest to the reference line.

4.2 More Sophisticated Approach

A more sophisticated approach, motivated by the hope of increasing the predictive power by using more than one model, is based on the observation that residuals still contain useful information that could be further used to improve accuracy. This idea of making use of residuals is inspired by the general principle that underlies boosting, with additional inspiration from stacking in terms of avoiding overfitting for the second level model.

The central idea of this approach is combining the predictions of two models, where the output of the first serves as a baseline and the second model provides a correction term that increases the overall predictive power. This can be achieved by using the second model to predict the residuals of the first. As a naïve approach, one may be tempted to directly use the residuals from the first model, which was fitted to the full training data, to train the second model. However, this would present a significant risk of overfitting, and thus a more elaborate procedure is required to train this model. Similarly, careful considerations have to be taken in regards to the hyperparameter tuning, not only to avoid overfitting, but also to minimise the impact of the more complex training procedure.

In principle, different combinations of models can be considered for this approach. That being said, it makes sense to select a relatively strong model as the first one, which acts as a stable baseline,

from which the second, more flexible model can learn the appropriate corrections. For this reason, a combination of a XGBoost for the first model and SVR for the second was deemed a suitable choice, taking into account their respective inherent properties.

There are further variations on this core idea that can be considered. For instance, the predictions of the first-level model may be passed as an additional input to the second-level, which may allow it to better learn the corrections that need to be applied. Furthermore, it can also be worthwhile to consider the case where this second model is trained on the true values rather than the residuals, while still receiving the predictions of the first model as an input.

4.2.1 Training the Model

The main complexity of the model training procedure stems from the need to avoid the overfitting that would arise from training the second model (SVR) on the values calculated directly from the predictions of the first (XGBoost). Similarly to stacking, where the second-level model is trained on out-of-fold predictions, the training of the SVR will be performed on the predictions obtained from data unseen by the XGBoost during the training phase. The most practical solution is to split the training data into separate folds, much like in k -fold cross-validation, and obtain out-of-fold residuals from XGBoosts trained on the remaining folds.

Let D denote the training data, and $m = 20$ the number of folds that will be considered for training. Training begins by partitioning the training data into folds $D_1, \dots, D_m$. Let B_t be the XGBoost trained on all folds except D_t ($t = 1, \dots, m$), and P_t represent the predictions of B_t obtained on the left-out-fold D_t . By iterating over all folds $t = 1, \dots, m$, a complete set of out-of-fold predictions P is obtained: $P = (P_1, \dots, P_m)$.

Following this step, the second-level model can be directly trained on all folds. Given the set of true values Y , and depending on the strategy in question, either the residuals R given by $R = P - Y$ or Y itself will be used as the target variable for the second level. On the other hand, if desired, the training input X may be modified into a $\tilde{X}$ by also including P as an input parameter, standardised by the mean and variance of Y . The first level model is obtained by training the XGBoost over the entire training data D , concluding the training and yielding the final model. Predictions for new data can be then obtained by combining the predictions of both levels. If the second-level is trained on the residuals, this will mean summing the predictions of both levels, otherwise this will correspond just to the second-level output. If the second-level model also receives the output of the first as an input parameter, then the predictions are scaled in the same way as during training.

4.2.2 Tuning the Model

To find the combination of hyperparameters which maximise the predictive power of the overall model, k -fold cross-validation is applied. It is worth noting, while the optimal combination of hyperparameters for the two models in this approach may differ from the hyperparameters resulting in optimal prediction for the

individual models, exploring the full dimensionality of the problem space when tuning is computationally demanding, to the point of being unfeasible. In order to mitigate this obstacle, a simplifying assumption was made. Given the inherent properties of the considered models, it is not entirely unreasonable to admit that the best hyperparameters for the XGBoost when tuned individually are close enough to the optimal choice in the combined model. In addition, if the hyperparameters for the first model are taken as fixed, the tuning of the second model can be further optimised by precalculating the relevant predictions. Therefore, the tuning procedure will focus on finding the best hyperparameters for the SVR, with an XGBoost with the same hyperparameters that were found to be optimal in Section 4.1 being used as the first model.

In order to apply k -fold cross-validation, it is clear that the data that will be used for tuning must be partitioned into k tuning folds. At the same time, as described in Section 4.2.1, the model training procedure itself already requires splitting the training data into m training folds, with each combination requiring an XGBoost to be fit in order to obtain the out-of-fold predictions. Furthermore, another XGBoost must be fit to all the training data, corresponding to $k - 1$ tuning folds, to provide the first-level predictions of the model. Finally, the SVR is trained on the out-of-fold predictions or residuals, and the MAE is calculated for the unseen tuning fold. As a result, a total of $k \times (m + 1)$ XGBoosts and k SVRs would need to be fitted in order to fully test a given combination of hyperparameters. However, given the assumption that the hyperparameters for the first level model are fixed, all the predictions of the XGBoosts are constant throughout the tuning, thus enabling a significant reduction of the computational cost by precalculating all the relevant results.

In the following, let $\Xi(V)$ denote the ordered set of predictions of a given model Ξ over the ordered set of measurements V , and let Y_V be the ordered sets containing the true response associated with the measurements in each point of V .

Let T denote the data that will be used for tuning, $k = 7$ the number of folds that will be considered for tuning, and $m = 20$ the number of folds that will be considered for training. Tuning begins by partitioning T into tuning folds $T_1, \dots, T_k$. Let C_i denote the set of data points such that all tuning folds except T_i are included, and let M_i be the overall model trained over C_i . The model is given by $M_i(V) = \varphi(B_i(V), S_i(\tilde{V}))$, where B_i and S_i are the XGBoost and SVR that are part of the model trained over C_i , respectively, $\tilde{V}$ is V possibly augmented with $B_i(V)$ if the SVR will take the predictions from the XGBoost as input and $\varphi(B_i(V), S_i(\tilde{V}))$ is either $B_i(V) + S_i(\tilde{V})$ or $S_i(\tilde{V})$, depending on whether the SVR is trained to predict the residuals or the temperatures.

While B_i is simply trained over all points of C_i , S_i relies on out-of-fold predictions. In order to obtain these predictions, C_i must be further partitioned into $F_{i,1}, \dots, F_{i,m}$ training folds. Let $G_{i,j}$ denote the set of data points such that all training folds except $F_{i,j}$ are included, and let $\tilde{B}_{i,j}$ be the XGBoost trained over $G_{i,j}$. The training of S_i requires calculating the out-of-fold predictions of the first model, P_i^{OOF} , given by:

$$P_i^{\text{OOF}} = (\tilde{B}_{i,1}(G_{i,1}), \dots, \tilde{B}_{i,m}(G_{i,m})).$$

The input for training S_i , $\tilde{C}_i$, will contain all the measurements in C_i , possibly augmented by P_i^{OOF} for every point. On the other hand, and given the definition of φ above, the target for training S_i can

be determined from taking $\varphi(P_i^{\text{OOF}}, S_i(\tilde{C}_i)) = Y_{C_i}$, which, by the Implicit Function Theorem, defines a specific function γ such that the previous equation becomes equivalent to $S_i(\tilde{C}_i) = \gamma(Y_{C_i}, P_i^{\text{OOF}})$. For the choices of φ that have been considered, $\gamma(Y_{C_i}, P_i^{\text{OOF}}) = Y_{C_i} - P_i^{\text{OOF}}$ when S_i is fitted to the residuals and $\gamma(Y_{C_i}, P_i^{\text{OOF}}) = Y_{C_i}$ when S_i is fitted to the true values.

Let also E be the expected prediction error estimated through cross-validation, which the tuning procedure aims to minimise. This can be expressed as

$$E = \frac{1}{k} \sum_{i=1}^k |Y_{T_i} - M_i(T_i)| = \frac{1}{k} \sum_{i=1}^k |Y_{T_i} - \varphi(B_i(T_i), S_i(\tilde{T}_i))| = \frac{1}{k} \sum_{i=1}^k |\gamma(Y_{T_i}, B_i(T_i)) - S_i(\tilde{T}_i)|,$$

where the T_i are the tuning folds.

Taking all of this into account, the tuning procedure of the overall model, which, as previously described, would require fitting $k \times (m + 1)$ XGBoosts and k SVRs for every combination of hyperparameters, can be reframed such that only k SVRs need to be fit, as long as the XGBoost is taken as fixed. This can be achieved by taking $\gamma(Y_{C_i}, P_i^{\text{OOF}})$ as the target for training the model on C_i and $\gamma(Y_{C_i}, B_i(T_i))$ as the target for calculating the MAE on the unseen tuning fold T_i .

Once the pairs of P_i^{OOF} and $B_i(T_i)$ have been pre-computed for all k combinations of tuning folds, any feasible strategy can be applied to the modified tuning of the SVR. For the purposes of the current project, the Optuna library was once again employed, in order to avoid the computational overhead of a more exhaustive manual grid search. All specifications regarding the tuning procedure remained the same as outlined previously for the basic models in Section 4.1.1, taking into account the different training and testing targets as explained in the present section.

4.2.3 Results

As before, the performance of the models is assessed using the MAE computed on both the training and test data. The results are presented in Table 4.2. Three models were considered: the most immediate case of training the SVR on the residuals from the XGBoost (Boosting + SVR); the case where the predictions of the first level are passed to the second level, which is still trained on the residuals (Boosting $\oplus$ SVR); and the case where the predictions are passed to the second level, but the latter is trained on the true values rather than the residuals (Boosting $\rightarrow$ SVR).

Table 4.2: Mean absolute error (MAE) on training and test sets across all time horizons for each model.

Model	12 hours		24 hours		48 hours	
	Train	Test	Train	Test	Train	Test
Boosting + SVR	0.480	1.485	0.403	1.695	0.593	2.297
Boosting $\oplus$ SVR	0.585	1.446	0.621	1.661	0.886	2.286
Boosting $\rightarrow$ SVR	0.669	1.434	0.907	1.675	1.199	2.283

Among the considered models in this subsection, the best performance in terms of the test MAE for the 24-hour prediction is achieved by the second approach (Boosting $\oplus$ SVR), whereas for the 12-hour

and 48-hour horizons, the third approach (Boosting $\rightarrow$ SVR) performs slightly better. The weakest performance for each time horizon is observed for the first approach (Boosting + SVR). However, it is worth noting that the differences between the worst- and best-performing models generally remain relatively small, particularly for the 48-hour predictions, where the gap is only 0.014.

It is also worthwhile to compare the three models considered in this subsection with the previously best-performing SVR (see Table 4.1). It is apparent that all of the more sophisticated approaches achieve improved performance, although the advantage for the first approach (Boosting + SVR) remains relatively small. In contrast, the differences between the best-performing two-stage model for each time horizon and SVR are much more noticeable, with the improvement of the third approach (Boosting $\rightarrow$ SVR) for the 12-hour and 48-hour predictions being around 0.06 and 0.04, respectively, and the improvement of the second approach (Boosting $\oplus$ SVR) for the 24-hour prediction horizon being around 0.05.

When it comes to the relationship between the training and test MAE, the gaps remain relatively large, especially for the first approach (Boosting + SVR). That being said, as all approaches perform relatively well, the extent of overfitting does not appear to be particularly problematic, and the models are effective at generalising on the unseen data.

In addition to assessing performance metrics, similarly to before, the patterns between the true and predicted temperatures across the three time horizons are also considered. The relationships for all models are shown in Figure 4.2. Overall, the same phenomenon as seen earlier is observed, where the models tend to underpredict at the upper tail and overpredict at the lower tail. The figure also confirms that the performance of the models is indeed similar, as the points lie generally very close to each other.

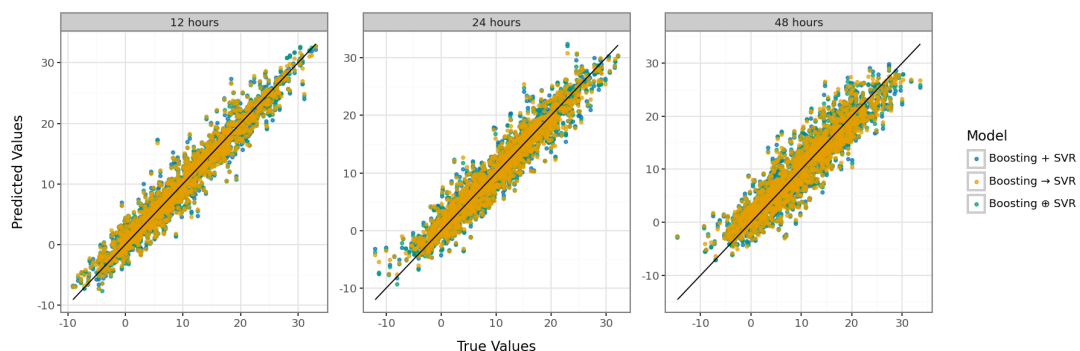


Figure 4.2: Relationship between true and predicted values for each model across 12-, 24-, and 48-hour horizons. The black line represents perfect predictions and serves as a reference.

4.3 Additional Approaches

The approaches that have been presented so far are already quite diverse, but it can be relevant to mention additional approaches that were explored, at least in part, and whose (preliminary) results did not suggest that significant improvements could be obtained. The initial experiments in order to

improve predictive accuracy considered taking simple averages of model predictions. Although this approach often yielded a slight improvement, it was not particularly substantial. Next, weighted sums of model predictions were considered, where it became apparent that optimising the weights should be done on the out-of-fold predictions, just like in stacking. Stacking itself was also briefly explored, but ultimately discarded from further analysis, in the hope of finding an approach that would use fewer models altogether.

In addition to the presented variations of combining XGBoost and SVR, one additional alternative was explored, which relies on a different strategy for training the second-level model, SVR. Specifically, instead of training a single model on the whole dataset, a different SVR was fitted to data from each season, which could, in theory, allow for a better modelling of some of the season-specific patterns. However, the results were subpar compared to the other approaches, which may be explained by the combination of a relatively small sample size for each season and the lack of granularity in the time information, making it impossible to properly handle measurements that, due to occurring near the beginning or end of a season, already display patterns of another.

5 Model Recommendation

The choice of which model to employ is highly dependent on the specific use case. If the intention is to simply get a first good approximation, simpler models are likely to be sufficient. Therefore, the recommendation would be to use SVR, as it achieved the strongest predictive performance across all time horizons among the models considered in Section 4.1. However, if the intention is to achieve the best possible prediction performance, attention should be turned towards the more sophisticated approaches involving the combination of XGBoost and SVR, as discussed in Section 4.2. Despite the the second option (Boosting $\oplus$ SVR) not achieving the absolute best predictive performance across all time horizons (for the 12-hour and 48-hour predictions, it was the third option (Boosting $\rightarrow$ SVR)), this approach is likely to be better in general, as fitting to residuals is expected to give a better generalisation compared to simply having the second model predict the true values and receive the first-level prediction as input. Furthermore, given that the MAEs between the two were quite comparable, it is possible that a more optimal choice of hyperparameters could further improve the predictive power of the second approach (Boosting $\oplus$ SVR) and allow it to perform better than the third option (Boosting $\rightarrow$ SVR). However, both of these approaches carry some computational complexity, further aggravated in both cases by the output of the first model being required to calculate the predictions of the second. In this sense, using the simpler approach, where both models are trained on the same input and SVR is trained to predict the residuals (Boosting + SVR), would allow these predictions to be obtained independently of each other, which could be desirable for some applications.

As the best-performing model for the 24-hour prediction horizon, the second approach (Boosting $\oplus$ SVR) was used to make the final prediction for the Kaggle competition. For this, the model was trained based on all the available data, as discussed before, and predictions were computed for the dataset `test`.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. DOI: <https://doi.org/10.1145/3292500.3330701>.
- Buck, A. L. (1981). New equations for computing vapor pressure and enhancement factor. *Journal of Applied Meteorology and Climatology*, 20(12):1527 – 1532. DOI: [https://doi.org/10.1175/1520-0450\(1981\)020<1527:NEFCVP>2.0.CO;2](https://doi.org/10.1175/1520-0450(1981)020<1527:NEFCVP>2.0.CO;2).
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 785–794. ACM. DOI: <http://doi.acm.org/10.1145/2939672.2939785>.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2 edition. DOI: <https://doi.org/10.1007/978-0-387-84858-7>.
- Lynch, P. (2008). The origins of computer weather prediction and climate modeling. *Journal of Computational Physics*, 227(7):3431–3444. DOI: <https://doi.org/10.1016/j.jcp.2007.02.034>.
- MathWorks (2025). Understanding support vector machine regression. URL: <https://www.mathworks.com/help/stats/understanding-support-vector-machine-regression.html>. Accessed: 24.11.2025.
- MeteoSwiss (2025a). Climate normals Bern/Zollikofen. Reference period 1991 – 2020. URL: <https://www.meteoswiss.admin.ch/climate/the-climate-of-switzerland/climate-normals/climate-diagrams-and-normals-per-station.html>. Accessed: 03.11.2025.
- MeteoSwiss (2025b). The climate of Switzerland. URL: <https://www.meteoswiss.admin.ch/climate/the-climate-of-switzerland.html>. Accessed: 16.12.2025.
- MeteoSwiss (2025c). Station information. URL: <https://www.meteoswiss.admin.ch/climate/the-climate-of-switzerland/records-and-extremes/extreme-value-analyses/background-information/station-information.html>. Accessed: 03.11.2025.
- National Centre for Atmospheric Science (NCAS) (2025). What is weather? URL: <https://ncas.ac.uk/learn/what-is-weather/>. Accessed: 02.11.2025.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830. URL: <http://jmlr.org/papers/v12/pedregosa11a.html>.
- Zhang, H., Liu, Y., Zhang, C., and Li, N. (2025). Machine learning methods for weather forecasting: A survey. *Atmosphere*, 16(1). DOI: <https://doi.org/10.3390/atmos16010082>.